

Linguagens de Programação

Aplicadas à Análise de Dados



Prof. Dr.

Felipe de Almeida Camargo



Curso

Inteligência e Análise de Dados





Objetivos da aula

- Compreender as estruturas condicionais (decisões) no Python
- Utilizar estruturas Condicionais;
- Compreender a lógica de listas;
- Praticar com exercícios.



Questionamentos

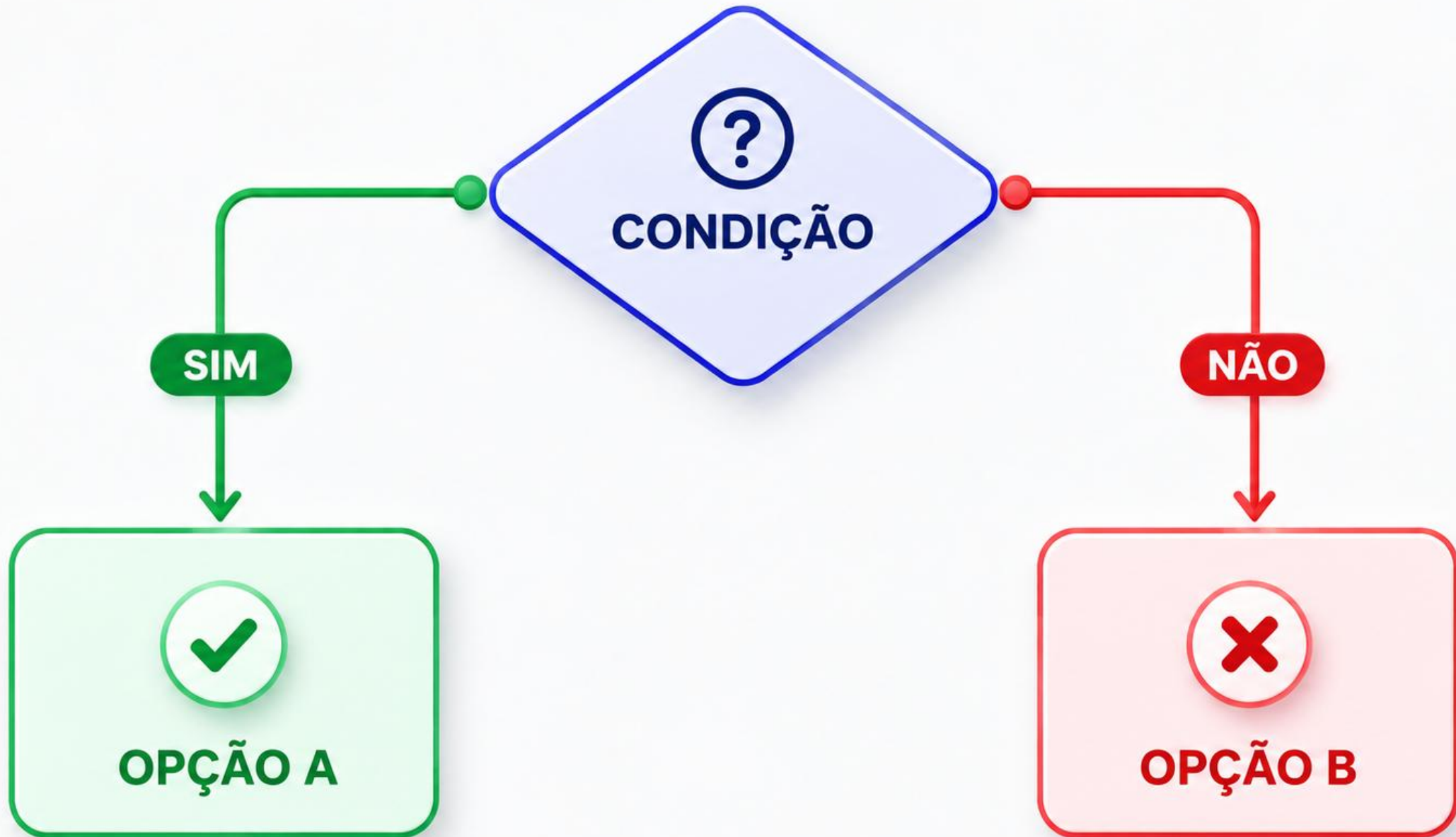
- Como um Banco aprova um empréstimo?
- Como um Hospital define um prioridade de atendimento?
- Como uma Plataforma de *streaming* recomenda filmes ao usuário?
- Como um Mercado sabe quando conceder um desconto ao cliente?

Respostas: Todos utilizam decisões (a partir de condições)!

O Computador sabe decidir?



Fluxograma de Decisão - Simples





O que é o comando if?

- O comando if permite que o programa execute um bloco de código somente se uma condição for verdadeira

```
1 if a > b:  
2     print("O valor de a é o maior!")
```

- **Observação:** A indentação é obrigatória em Python para que o bloco seja executado corretamente.

Prática: Vocês são responsáveis por perguntar a temperatura atual ao usuário e se a temperatura for acima de 30°C exibir uma mensagem de se hidratar!

```
1 temperatura_atual = float(input("Qual é a temperatura atual: "))  
2  
3 if temperatura_atual >= 30:  
4     print("Cuidado! a temperatura atual é superior à 30°C, recomenda-se hidratar!")
```



If e else

Situação: Um sensor mede a temperatura de uma máquina. **Se** a temperatura for acima de 80°C, exiba um alerta. **Caso contrário**, exiba que a máquina está normal.

Nem sempre queremos executar algo apenas quando a **condição** é **verdadeira**. Precisamos de uma ação para **casos contrários**.

```
1 if a > b:  
2     print("O valor de a é o maior!")  
3 else:  
4     print("O valor de b é o maior!")
```

Se a primeira **condição** for falsa (bloco 1), o **else** (caso contrário) é executado.

Prática: Desenvolva um programa que solicite ao usuário um **número inteiro** e determine se o número informado é **PAR** ou **ÍMPAR**.



Exemplo prático

Prática: O sistema recebe a informação da senha do usuário. Se o usuário digitar corretamente a senha do sistema, o acesso é permitido. Caso contrário, o acesso deve ser negado.

```
1 senha = int(input("Digite sua senha:"))
2 if senha == 1234:
3     print("Acesso permitido")
4 else:
5     print("Acesso negado! Senha incorreta")
```

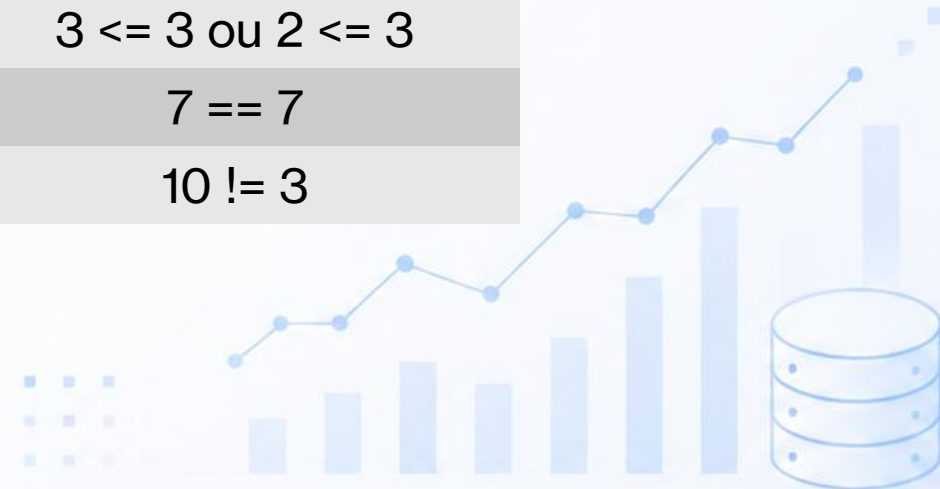




Operadores Relacionais

Esse tipo de operadores (relacionais) são utilizados para comparação.

Operador	Significado	Exemplo
>	Maior que	5 > 2
<	Menor que	3 < 4
>=	Maior ou igual	5 >= 5 ou 6 > 5
<=	Menor ou igual	3 <= 3 ou 2 <= 3
==	Igual	7 == 7
!=	Diferente	10 != 3





Exemplos com *strings*

```
1 nome = "Felipe"
2
3 print(nome == "José") # False
4 print(nome != "Bethina") # True
5 print(nome > "Bethina") # True
6 print(nome < "Arnaldo") # False
7 print(nome > "Bethina") # True
```

Obs.: Em Python, podemos comparar *strings* considerando a ordem lexicográfica (ordem alfabéticas).



Valores Booleanos

Em Python, uma comparação produz um **valor lógico**, chamado de **booleano**.

- Existem apenas dois valores booleanos: **True** (verdadeiro) e **False** (falso)

As estruturas condicionais utilizam esses resultados (**True** ou **False**) para decidir qual bloco de código será executado.

```
1 idade = 20
2
3 print(idade >= 18) # True
4 print(idade < 18) # False
```




Três ou mais possibilidades - elif

O comando elif permite verificar outras condições caso a primeira seja falsa;

É usado em conjunto com os comandos if e else (entre eles)

```
1 nota = float(input("Digite sua nota: "))
2
3 if nota >= 7:
4     print("Parabéns! Você foi Aprovado!")
5
6 elif nota >= 5:
7     print("Você está de Recuperação!")
8
9 else:
10    print("Infelizmente você foi Reprovado!")
```





Exemplo prático

```
1 nota = float(input("Nota: "))
2
3 if nota >= 9:
4     print("Sua nota é Excelente. Sua classificação é: Avançado!")
5
6 elif nota >= 7:
7     print("Sua nota é Boa. Sua classificação é: Intermediário")
8
9 elif nota >= 5:
10    print("Sua nota é Regular. Sua classificação é: Básico!")
11
12 else:
13    print("Sua nota é Ruim! Sua classificação é: Abaixo do Básico!")
```





Operadores Lógicos

Outras condições:

Operador	Significado	Exemplo
and	e (todas verdadeiras)	$a > 0$ and $b > 2$
or	ou (pelo menos uma verdadeira)	$a > 0$ or $b > 1$
not	não (inverte)	not ($a > 0$)



Exemplo com and

```
1 idade = 20
2 renda = 2500
3
4 if idade >= 18 and renda >= 2000:
5     print("Pode financiar!")
6
7 else:
8     print("Não pode financiar.")
```

Ambas as condições precisam ser verdadeiras.



```
1 idade = 25
2 renda = 3000
3 sem_dividas = True
4
5 if idade >= 18 and renda >= 2000 and sem_dividas:
6     print("Financiamento aprovado")
7
8 else:
9     print("Financiamento não aprovado")
```

Todas as condições precisam ser verdadeiras.





Exemplo com or

```
1 tem_cracha = True
2 visitante = True
3
4 if tem_cracha or visitante:
5     print("Acesso liberado")
6
7 else:
8     print("Acesso negado")
```

```
1 idade = 35
2 valor_compra = 650
3 compras_anteriores = 2
4
5 if idade >= 60 or valor_compra >= 500 or compras_anteriores >= 5:
6     print("Desconto concedido")
7
8 else:
9     print("Sem desconto")
```





Exemplo com not

```
1 bloqueado = False
2
3 if not bloqueado:
4     print("Usuário ativo")
5
6 else:
7     print("Usuário bloqueado")
```

Inverte a condição lógica





Prática com os três comandos (and, or, not)

Situação: Aprovação de uma compra. Uma loja possui um sistema que decide se uma compra pode ser aprovada. A compra será aprovada quando:

- o cliente tiver **18 anos ou mais e** renda de pelo menos **R\$ 2.000**;
- **ou** o cliente for **VIP**;
- além disso, o cliente **não pode estar bloqueado**.

```
1 idade = 20
2 renda = 2500
3 cliente_vip = False
4 bloqueado = False
5
6 if (idade >= 18 and renda >= 2000 or cliente_vip) and not bloqueado:
7     print("Compra aprovada")
8
9 else:
10    print("Compra não aprovada")
```





E quando temos vários dados?

Até aqui, nossos programas recebem, analisam e tomam decisões sobre **valores individuais**.

Exemplo:

```
1 temperatura = 82.5  
2 venda = 1500  
3 nota = 8.5
```

Mas, na **Análise de Dados**, normalmente trabalhamos com apenas um valor?



Não. Trabalhamos com conjuntos de dados.



1 VALOR



x = 1250

VÁRIOS VALORES

- 1250
- 980
- 1430
- 760
- 1520
- ...

ANÁLISE



Como vocês acham que o Python consegue guardar vários valores juntos?



Listas em Python

Em Python, uma **lista** permite armazenar vários valores em uma única variável.

Podemos criar uma lista diretamente:

```
1 vendas = [1250, 980, 1430, 760, 1520]
```

Os colchetes **[]** indicam a **lista** e os **elementos** são separados por **vírgulas**.

Podemos começar com uma lista vazia: **valores = []**

E adicionar dados posteriormente:

```
1 valores = []  
2  
3 valores.append(10)  
4 valores.append(25)  
5  
6 valores # [10, 25]
```





Listas em Python – Método .append()

.append() é um **método das listas** utilizado para adicionar um elemento ao final da lista.

```
1 nome_da_lista.append(valor)
```

O **ponto** indica que estamos utilizando uma operação que **pertence àquele objeto**. **nome_da_lista** é uma lista e **.append()** é um método disponível para listas.





Prática: armazenando vários dados

- Uma empresa deseja registrar o faturamento dos últimos **cinco dias úteis**.

```
venda1 = float(input("Venda de segunda-feira: "))  
venda2 = float(input("Venda de terça-feira: "))  
venda3 = float(input("Venda de quarta-feira: "))  
venda4 = float(input("Venda de quinta-feira: "))  
venda5 = float(input("Venda de sexta-feira: "))
```

Precisamos realmente criar uma variável diferente para cada valor?

```
vendas = []  
  
vendas.append(float(input("Venda de segunda-feira: ")))  
vendas.append(float(input("Venda de terça-feira: ")))  
vendas.append(float(input("Venda de quarta-feira: ")))  
vendas.append(float(input("Venda de quinta-feira: ")))  
vendas.append(float(input("Venda de sexta-feira: ")))  
  
print(vendas)
```

Com o método `.append()` adicionamos um novo elemento ao final da lista.





Quantos dados temos? len()

Situação: Uma indústria registrou a produção dos últimos dias: 120, 135, 128, 142, 150. Precisamos descobrir **quantos registros foram coletados**.

```
1 producao = [120, 135, 128, 142, 150]
2
3 quantidade = len(producao)
4
5 print(f"Quantidade de registros: {quantidade}")
```

O comando **len()** retorna a quantidade de elementos existentes em uma lista.

Prática: Acrescente a lista de produção mais 4 registros, usando `.append()` (manualmente ou por input) e verifique novamente a quantidade de registros usando o comando `len()`.





Maior e menor valor max() e min()

Agora a empresa quer identificar os **extremos de sua produção**. E seu papel é encontra-los.

```
1 producao = [120, 135, 128, 142, 150]
2 maior = max(producao)
3 menor = min(producao)
4 print(f"Maior produção: {maior} e a Menor produção: {menor}")
```





Total produzido e média `sum()` + `len()`

Continuando com os mesmos dados. Queremos saber **quanto foi produzido no período** e a **produção média diária**.

```
1 producao = [120, 135, 128, 142, 150]
2
3 total = sum(producao)
4 media = total / len(producao)
5
6 print(f"Produção total: {total} e a média: {media:.2f}")
7
```





Acessando os dados: índices

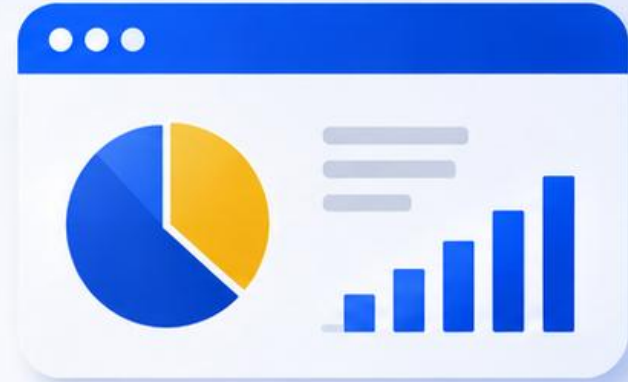
Além de analisar a lista inteira, podemos acessar **um dado específico**.

Valor	120	135	128	142	150
Índice	0	1	2	3	4
	Segunda	Terça	Quarta	Quinta	Sexta

```
1 print(producao[0])  
2 print(producao[2])  
3 print(producao[4])
```

- Obs.: Em Python, o primeiro elemento de uma lista possui **índice 0**.

Problemas Aplicados



Problema 1 – Desempenho de uma Rede de Supermercados

Situação: Uma rede de supermercados registrou o faturamento diário de uma de suas unidades durante cinco dias úteis, ver Tabela.

A direção estabeleceu que uma unidade apresenta:

- **Bom desempenho:** média diária igual ou superior a R\$ 22.000;
- **Desempenho regular:** média entre R\$ 19.000 e R\$ 21.999,99;
- **Desempenho crítico:** média inferior a R\$ 19.000.

Desenvolva um programa que permita analisar os dados da unidade e apresente:

- quantidade de dias analisados;
- faturamento total;
- faturamento médio diário;
- maior faturamento registrado;
- menor faturamento registrado;
- classificação do desempenho da unidade conforme os critérios da direção.

Dia	Faturamento
Segunda	R\$ 18.500
Terça	R\$ 21.300
Quarta	R\$ 17.800
Quinta	R\$ 24.600
Sexta	R\$ 27.400



Problema 2 – Controle de Velocidade

Situação: Um radar registrou a velocidade de um veículo em uma via cujo limite é de **70 km/h**.

Desenvolva um programa que:

- Leia a **velocidade do veículo**;
- Verifique se ele ultrapassou **70 km/h**;
- Informe se o veículo está **dentro do limite** ou se o motorista **foi multado**;
- Calcule a multa considerando **R\$ 9,00 por km/h excedente**;
- Exiba o **valor total da multa**.
- Calcule o **percentual de excesso de velocidade** em relação ao limite.

$$\text{Excesso percentual} = \frac{\text{Velocidade} - \text{Limite}}{\text{Limite}} \times 100$$

Problema 3 – Monitoramento de Produção Industrial

Situação: Uma indústria possui uma linha com capacidade planejada de **500 unidades por turno**. Nos últimos seis turnos, foram registradas as seguintes produções: **485, 510, 472, 498, 455 e 520** unidades. A equipe utiliza a **produção média** para avaliar o desempenho geral do período, ver Tabela.

Além da classificação geral, existe uma regra de segurança operacional: Se **pelo menos um turno** apresentar produção inferior a **460 unidades**, deverá ser emitido um **alerta operacional**, independentemente da classificação obtida pela média.

Desenvolva um programa que apresente:

- armazene as produções em uma lista;
- determine a quantidade de turnos analisados;
- calcule a produção total do período;
- calcule a produção média por turno;
- identifique a maior e a menor produção registrada;
- classifique o desempenho geral do período;
- verifique se deve ser emitido um **alerta operacional**.

Produção média	Situação
≥ 500	Meta atingida
480 a 499,99	Atenção
< 480	Abaixo da meta



Problema 4 – Análise de Rentabilidade de uma Empresa

Uma empresa deseja avaliar o desempenho financeiro de suas operações. O setor financeiro disponibilizou os dados dos últimos cinco meses:

Dados disponíveis			
Mês	Receita	Custos Variáveis	Custos Fixos
Janeiro	R\$ 85.000	R\$ 46.000	R\$ 18.000
Fevereiro	R\$ 92.000	R\$ 51.000	R\$ 19.000
Março	R\$ 78.000	R\$ 48.000	R\$ 20.000
Abril	R\$ 105.000	R\$ 54.000	R\$ 21.000
Maior	R\$ 88.000	R\$ 52.000	R\$ 22.000

Para avaliar cada mês, a empresa calcula inicialmente o **resultado financeiro da operação**:

Resultado = Receita – Custos Variáveis – Custos Fixos

Em seguida, é calculado o **Índice de Margem Operacional (IMO)**:

IMO = (Resultado / Receita) x 100

A diretoria utiliza os seguintes critérios para avaliar o desempenho:

Índice de Margem Operacional	Classificação
IMO ≥ 25%	Excelente
15% ≤ IMO < 25%	Bom
8% ≤ IMO < 15%	Regular
0% ≤ IMO < 8%	Atenção
IMO = 0%	Ponto de equilíbrio
IMO < 0%	Prejuízo

Desafio: Os dados apresentados na tabela representam cinco meses de operação da empresa. Para realizar a análise, organize os dados utilizando listas.

Desenvolva um programa em Python que:

- a) Crie cinco listas correspondentes aos meses, onde para **cada mês**, armazenando as informações na seguinte ordem: **[Receita, Custos Variáveis, Custos Fixos]**.
- b) utilize os índices das listas para acessar os dados necessários de cada mês;
- c) calcule o **resultado financeiro** de cada mês;
- d) calcule o **Índice de Margem Operacional (IMO)** de cada mês;
- e) classifique individualmente o desempenho financeiro de cada mês de acordo com os critérios definidos pela diretoria;
- f) armazene os cinco valores de IMO calculados em uma nova lista;
- g) identifique o **maior IMO** e o **menor IMO** obtidos no período;
- h) determine a qual mês correspondem o melhor e o pior desempenho;
- i) apresente um relatório contendo, para cada mês, o resultado financeiro, o IMO e sua respectiva classificação.

Atividades

Obrigado!

